

VHDL Reference

An extensive reference guide for VHDL basics that includes essential language framework, data types, operators, and primary programming constructs, including code examples.

Electronic Design, Department of Electronic, University of Alcalá
carlos.cruz@uah.es
Date: 25/10/2025

VHDL is a case insensitive and strongly typed language

1. Compilation Units

Every VHDL design file consists of three primary parts: Library, Entity, and Architecture.

- Library Usage Declarations
- Entity Declarations
- Architecture Declarations
- Package Declarations
- Configuration Declarations

2. Entity Declaration = IO of a design

The entity defines the public interface of your circuit - its input and output ports. It's where you declare all your design's external variables. The semicolon acts as a separator, with no semicolon on the last port.

```
entity n_input_nand is generic (n : integer := 2);  
    port (data : in bit_vector(1 to n);  
          result : out bit );  
  
end n_input_nand  
-- port directions: in | out | inout | buffer | linkage
```

3. RTL Architecture = Implementation

Describes the internal behavior and logic of an entity. This is where you define how the outputs are derived from the inputs. RTL code creates hardware and/or logic. RTL code contains assignments and process statements

```
architecture behave of n_input_nand is  
    -- declarations  
begin  
    -- concurrent statements  
end behave;
```

4. VHDL Operators

logical operators : **and, or, xor, nand, nor, xnor, not**
relational operators : =, /=, <, <=, >, >=
shift left/right logical operators: **sl, srl**
shift left/right arithmetic operators: **sla, sra**
rotate left/right logical operators: **rol, ror**
other operators: +, -, &, *, **, /, **mod, abs, rem**
eg. &: concatenation, '1' & "10" = "110"
****:** exponentiation, 2 ** 3 = 8
rem: remainder, 7 rem 2 = 1
mod: division modulo, 5 mod 3 = 2

5. Data Types

VHDL strongly typed. STD_LOGIC the flexible type for digital design

Type	Description	Values	Library Required
BIT	Simple binary value '0' and '1' only		std.standard
STD_LOGIC	Enhanced logic '0', '1', 'Z', 'X', 'U' IEEE.	STD_LOGIC_1164	
STD_LOGIC_VECTOR	Array of STD_LOGIC "110" STD_LOGIC_1164		

Vectors: Arrays of a base type. Range can be DOWNTO (descending) or TO (ascending).

```
SIGNAL my_bus : STD_LOGIC_VECTOR(7 DOWNTO 0);  
SIGNAL data : STD_LOGIC_VECTOR(0 TO 7);  
my_bus <= "10101100";  
my_bus <= x"AC";  
my_bus <= (7=>'1', 3=>'1', OTHERS=>'0');
```

A) Predefined Data Types
bit
'0' and '1' bit_vector
Array of "bit" boolean
true and false

Character	7-bit ASCII
integer	signed 32 bit at least
natural	integer >= 0
positive	integer > 0
real	Floating point, min : +1e38 to -1e38
string	Array of characters
time	hr, min, sec, ms, us, ns, ps, fs

B) User Defined Data Types

```
type range is range 0 to 100000  
units  
meter; -- base unit  
kilometer = 1000 meter;  
end units distance;  
type number is integer;  
type voltage is range 0 to 5;  
type current is range 1000 downto 0;  
type d_bus is array ( range <=> ) of bit;  
type instruction is record  
    opcode : bit; operand : bit;  
end record;  
type int_file is file of integer;  
type pointer_to_integer is access integer;  
subtype positive_number is integer range 0 to 100000  
type fourval is ( X, L, H, Z );  
subtype resolve_n is resolve twoval;
```

C) Working with Numbers

Arithmetic operations like addition and subtraction can only be performed on data of the same type. A STD_LOGIC_VECTOR must be type-cast to a numeric type like signed or unsigned before you can perform math on it. For this, you must use the IEEE.NUMERIC_STD.ALL library.

```
SIGNAL a, b, sum : STD_LOGIC_VECTOR(7 DOWNTO 0);  
sum <= STD_LOGIC_VECTOR(UNSIGNED(a) + UNSIGNED(b));  
sum <= STD_LOGIC_VECTOR(SIGNED(a) + SIGNED(b));  
result <= STD_LOGIC_VECTOR(UNSIGNED(data) + 1);  
result <= STD_LOGIC_VECTOR(UNSIGNED(data) - 1);  
result <= STD_LOGIC_VECTOR(UNSIGNED(data) SLL 1);
```

D) Custom Data Types

VHDL allows to define your own data types for better clarity and modeling. Useful for modeling memories. A custom type to model a small memory with 256 entries. Each entry is an integer from 0 to 255.

```
TYPE memory_array IS ARRAY(0 TO 255) OF INTEGER RANGE 0 TO 255;  
TYPE state_type IS (IDLE, ACTIVE, ERROR, RESET);  
SIGNAL my_memory : memory_array;  
SIGNAL current_state : state_type := IDLE;  
my_memory(10) <= 255;  
data_out <= my_memory(addr);
```

6. Declarations

```
constant bus_width : integer := 32  
variable read_flag : bit := 0;  
-- only in processes  
-- and subprograms  
signal clock " bit;  
file f3 : int_file open write_mode is "test.out";  
alias enable : bit is addr(31);  
attribute delay : time;  
component n_input_nand  
    generic ( n : integer := 2 );  
    port ( data : in bit_vector ( 1 to n ); result : out bit );  
end component n_input_nand;  
function square ( i : integer ) return integer;  
for store : use configuration latch;
```

7. Attributes

```
-- type my_array is array ( 9 downto 0 ) of any_type;  
-- variable an_array : my_array;  
-- type fourval is ( '0', '1', 'Z', 'X' );  
-- signal sig : sigtype;  
-- constant T : time := 10 ns;
```

Attribute	Result type	Result
my_array'high	any_type	9
my_array'left	any_type	9
my_array'low	any_type	0
my_array'right	any_type	0
my_array'ascending	boolean	false
my_array'length	integer	10
my_array'range	integer	9 downto 0
my_array'reverse_range		0 to 9
integer		
fourval'leftof('0')	fourval	error
fourval'leftof('1')	fourval	'0'
fourval'pos('Z')	integer	2
fourval'pred('1')	fourval	'0'
fourval'rightof('1')	fourval	'Z'
fourval'succ('Z')	fourval	'X'
fourval'val(3)	fourval	'X'
sig'active	boolean	

True if activity on sig
sig'delayed(T)
Copy of sig delayed by T
sig'driving_value
Value of driver on sig
sig'event
True if event on sig
sig'last_active
Time since last activity
sig'last_event

Time since last event	
sig* last_value	sigtype
Value before last event	
sig* quiet(T)	boolean
Activity (now – T) to now	
sig* stable(T)	boolean
Event (now – T) to now	
sig* transactio	bit
Toggles on activity on sig	

8. Assigning Values

```

A_sl <= '1' ; -- Character literal
B_slv <= "1111" ; -- string literal
C_slv <= X"F" ; -- hex. 4 bits per character
E_slv <= (others => '1') ; -- aggregate
L_int <= 15 ; -- universal integer
M_int <= 16#F# ; -- base literal (16 = base)
N_bool <= TRUE ; -- boolean only true or false

```

9. Modeling Styles: Structural vs Behavioral

You can implement logic in two main ways: **Structural Modeling**, which describes a circuit by connecting instances of pre-existing components. It focuses on the netlist or schematic view of the hardware. Structural code connects lower levels of a design. Structural code has three pieces: component declarations, signal declarations, and component instances (creates the connectivity).

```

architecture Structural of MuxReg is -- Component Declarations
component Mux8x2
port (
    Sel : In std_logic ;
    I0, I1 : In unsigned(7 downto 0);
    Y : Out unsigned(7 downto 0)
);
end component ;
component Reg8
port (
    Clk : In std_logic ;
    D : In unsigned(7 downto 0);
    Q : Out unsigned(7 downto 0)
);
end component ;
-- Signal Declarations
signal Mux : unsigned(7 downto 0);
begin
-- Component Instantiations
-- Named Association
Mux8x2_1: Mux8x2
port map (
    Sel => Sel,
    I0 => A,
    I1 => B,
    Y => Mux
);
-- Positional Association
Reg8_1: Reg8
port map (Clk, Mux, Y);
end Structural;

```

Behavioral Modeling: Describes the circuit's behavior directly using logic equations and processes. It focuses on the flow of

```

data and the algorithm.
ARCHITECTURE behavioral OF multiplexer IS
BEGIN
    output <= a WHEN sel = '0' ELSE b;
END ARCHITECTURE;

ARCHITECTURE structural OF multiplexer IS
    COMPONENT and_gate PORT (x, y : IN STD_LOGIC; z : OUT STD_LOGIC);
    COMPONENT or_gate PORT (x, y : IN STD_LOGIC; z : OUT STD_LOGIC);
    SIGNAL not_sel, and1_out, and2_out : STD_LOGIC;
BEGIN
    U1: and_gate PORT MAP (a, not_sel, and1_out);
    U2: and_gate PORT MAP (b, sel, and2_out);
    U3: or_gate PORT MAP (and1_out, and2_out, output);
    not_sel <= NOT sel;
END ARCHITECTURE;

```

Generics: Creating Reusable Components

A **generic** is a parameter you can pass to an entity to make it more flexible, like a global constant for that instance. This allows you to create configurable components.

```

ENTITY n_bit_register IS
    GENERIC ( N : INTEGER := 8 );
    PORT (
        d : IN STD_LOGIC_VECTOR(N-1 DOWNT0 0);
        q : OUT STD_LOGIC_VECTOR(N-1 DOWNT0 0)
    );
END ENTITY n_bit_register;

```

10. Concurrent Statements: coded in the architecture.

A) Signal Assignments: Expression is evaluated immediately. Value is assigned one delta cycle later.

B) Simple Assignment =logic and/or wires

```

Z <= AddReg ;
Sel <= SelA and SelB ;
YL <= A(6 downto 0) & '0'; --Shift Lt
YR <= '0' & A(7 downto 1); --Shift Rt
SR <= SI_sl & A(7 downto 1); --Shift In

```

C) Conditional Assignment

```

Mux2 <=
    A when (Sel1 = '1' and Sel2 = '1')
    else B or C ;
ZeroDet <= '1' when Cnt = 0 else '0';

```

D) Selected Assignment

```

with MuxSel select
Mux41 <= A when "00", B when "01", C when "10",
    D when "11", 'X' when others;

```

E) Process = Container of Sequential Code

Must have either a sensitivity list or wait statement. Combinational logic requires all inputs (signals read in the process) to be on the sensitivity list. The "is" following the sensitivity list is optional.

```

Mux : process (MuxSel, A, B, C, D) is
begin

```

```

    case MuxSel is
    when "00" => Y <= A ;
    when "01" => Y <= B ;
    when "10" => Y <= C ;
    when "11" => Y <= D ;

```

```

        when others => Y <= 'X';
    end case;
end process;

```

```

state_mach: process ( state ) -- label is optional
-- variable declarations
begin
-- sequential statements
end process;

```

```

U1_n_input_nand : n_input_nand
generic map ( n => 2 )
port map ( data => my_data;
    result => my_res );

```

```

top_block : block
-- declaration

```

```

begin
-- concurrent statements
end block;

```

```

    label1: for i in 1 to 3 generate
label2: nand2( a(i), b(i), c(i) );
end generate
label3: if ( i < 4 ) generate
label4: nor2( a(i), b(i), c(i) );
end generate;

```

11. Sequential Statements: Contained in processes and subprograms

A) Signal Assignment

```

Z <= AddReg;
Sel <= Sel1 and Sel2;

```

VHDL-2008 allows conditional and selected assignments in sequential statements

B) Variable Assignment

Expression is evaluated and assigned immediately.

```

MuxSel := S1 & S0;

```

C) IF Statement

```

if (in1 = '1') then
    NextState <= S1 ; Out1 <= '1' ;
elsif (in2 = '1' and in3 = '1') then
    NextState <= S2 ;
elsif (in4 and in5) = '1' then
    NextState <= S3 ;
else
    NextState <= S4 ;
end if ;

```

An IF statement can have one or more signal assignments per branch. Prior to VHDL-2008, the conditional expression must be boolean. With VHDL- 2008 it may also be bit or std_ulogic (std_logic).

D) Case Statement

```

Mux : process (S1, S0, A, B, C, D)
variable MuxSel :
std_logic_vector(1 downto 0) ;
begin
MuxSel := S1 & S0 ;
    case MuxSel is
        when "00" => Y <= A;

```

```

when "01" => Y <= B;
when "10" => Y <= C;
when "11" => Y <= D;
when others => Y <= 'X';

end case ;
end process ;

```

- E) Asynchronous Reset Flip-Flop.** Asynchronous reset is specified before the clock. Clock and reset must be on the sensitivity list.

```

RegProc : process ( Clk, nReset)
begin
if (nReset = '0') then
    AReg <= '0';
    BReg <= '0';
elsif rising_edge(Clk) then
    if LoadEn = '1' then
        AReg <= A;
        BReg <= B;

    end if;
end if;
end process;

```

- F) Synchronous Reset Flip-Flop.** Synchronous reset is specified after the clock. Only clock must be on the sensitivity list.

```

RegProc: process (Clk)
begin
    if rising_edge(Clk) then
        if (nReset = '0') then
            AReg <= '0';
        elsif LoadEn = '1' then
            AReg <= A;
        end if;
    end if;
end process;

```

- G) For Loop:** Loop index does not need to be declared. For synthesis, loop index must be integer.

```

RevAProc : process(A)
begin
    for i in 0 to 7 loop
        RevA(7 - i) <= A(i);
    end loop;
end process;

```

- H) Creating Clock**

```

Clk1 <= not Clk1 after 10 ns;
ClkProc : process
begin
    Clk2 <= '0'
    wait for 10 ns;
Clk2 <= '1';
    wait for 10 ns;
end process;

```

- I) Wait Until and after:** Wait stops a process for at least a delta cycle. Wait until Clk = '1' finds the next rising edge of clock and is used extensively in testbenches. Signal assignments using "after" always project a value on a signal. "After" never causes a process to stop.

```
TestProc : process begin
```

```

wait until Clk = '1';
Addr <= "000" after tpd_Clk_Addr;
wait until Clk = '1';
Addr <= "001" after tpd_Clk_Addr;
-- and so on ...
wait for tperiod_clk * 5;
report "Test Done" severity failure;
end process;

```

12. Generate Statement

The GENERATE statement acts like a for loop for creating multiple instances of concurrent hardware statements, perfect for repetitive structures.

```

ARCHITECTURE structural OF parallel_adder IS
BEGIN
    gen_label: FOR i IN 0 TO 7 GENERATE
        and_instance: ENTITY work.and_gate PORT MAP (a(i), b(i), c(i));
    END GENERATE gen_label;
END ARCHITECTURE;

```

13. Component Instantiation

This is the process of using a pre-defined entity as a component in another design. Each instance is a unique copy and must be given a unique label to differentiate it.

```

COMPONENT counter IS
    PORT (
        clk : IN STD_LOGIC;
        reset : IN STD_LOGIC;
        count : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
    );
END COMPONENT;
U_Counter: counter PORT MAP (
    clk => system_clock,
    reset => system_reset,
    count => counter_output );

```

14. Concurrent and Sequential Statements

```

enable <= select after 1 ns;
assert ( a = b )
report "a is not equal to b"
severity note;
-- severity levels : note | warning | error | failure

```

15. Package Declarations

```

package two_level is
-- type, signal, functions declarations
end two_level;
package body two_level is
-- subprogram definitions
end two_level;

```

16. Library Usage Declarations

```

-- using the two_level package.
library work;
use work.two_level.all;           -- all objects used
use work.two_level.vcc;          -- only the "vcc" object used

```

Subprograms

```

function bool_2_level ( boolean : in_bool )
return two_level is
variable return_val : two_level;

```

begin

```

if ( in_bool = true ) then
    return_val := high;
else
    return_val := low;
end if;
return return_val;
end bool_2_level;

```

procedure clock_buffer

```

( signal local_clk: inout bit;
signal clk_pin: in bit;
constant clock_skew: in time ) is
begin -- example of side effects in a procedure
global_clk <= local_clk
after clk_skew; local_clk <= clk_pin;
end clock_buffer;

```

17. Predefined Subprograms

```

enable <= '1' when ( now < 2 ns ) else '0';
variable ptoi : pointer_to_integer;
ptoi := new integer; -- usage of new deallocate (ptoi);
variable status : file_open_status; file my_file : int_file;
file_open( status, my_file, "in.dat", read_mode );
end_file ( my_file ); -- returns true/false
variable int_var : integer;
read ( my_file, int_var );
file_close ( my_file );

```

18. Configuration Declarations

```

configuration input_8 of n_nand is for customizable
for a1 : nand_2
use entity work.nand_2 ( n_nand_arch );
end for; end for;
end input_8;

```

11.

19. Non-synthesizable Constructs

Most tools will not synthesize : access, after, alias, assert, bus, disconnect, file, guarded, inertial, impure, label, linkage, new, on, open, postponed, pure, reject, report, severity, shared, transport, units, with.

20. Standard Packages

A) Common Packages

```

Usage Abbr. Source
use std.standard.all ; -- * std IEEE
ieee.std_logic_1164.all ; 1164 IEEE
use ieee.numeric_std.all ; ns IEEE
use ieee.numeric_std_unsigned.all; nsu IEEE
use ieee.std_logic_arith.all ; sla Shareware
use ieee.std_logic_unsigned.all ; slu Shareware
use std.textio.all ; textio IEEE
use ieee.std_logic_textio.all ; - Shareware
VHDL-2008 adds packages for fixed and floating point.

```

B) Common Synthesizable Types

```

Type / Abbreviation Value Package
std_logic / sl U X 0 1 Z W L H - 1164
std_logic_vector / slv array of std_logic 1164

```

signed / sv array of std_logic ns, sla
 unsigned / uv array of std_logic ns, sla
 boolean / bool (False, True) std
 integer / int -(2³¹ - 1) to 2³¹ - 1 std
 natural / int0+ 0 to 2³¹ - 1 std
 line access string textio
 Enumerated type StateType is (S0, S1, S2, S3) ;

C) IEEE.STD_LOGIC_1164 Package

type **std_ulogic** is ('U', 'X', '0', '1', 'W', 'L', 'H');
 type **std_ulogic_vector** is array
 (natural range <>) of std_ulogic;
 function **resolved** (s : std_ulogic_vector) return std_ulogic; subtype
std_logic is resolved std_ulogic;
 type **std_logic_vector** is array
 (natural range <>) of std_logic;
 function **to_bit** (s : std_ulogic; xmap : bit := '0') return bit;
 function **to_bitvector**: std_logic_vector; xmap : bit := '0')
 return bitvector;
 function **stdlogicvector** (b:bit_vector) return std_logic_vector;
 function **rising_edge** (signal s : std_ulogic) return boolean;
 function **falling_edge** (signal s : std_ulogic) return boolean;
 function **is_x** (s : std_logic_vector) return boolean;

D) STD.TEXTIO Package

type **line** access string;
 type **text** is file of string;
 type **side** is (right, left);
 subtype **width** is natural;
 file **input** : text open read_mode is "std_input";
 file **output** : text open write_mode is "std_output";
 procedure **readline** (file f : text; l : out line);
 procedure **writeline** (file f : text; l : in line);
 procedure **read** (l: inout line;
 value: out bit;
 good: out boolean);
 procedure **write** (l: inout line; value: in bit;
 justified: in side := right;
 field: in width := 0);

-- The type of "value" can be bit_vector | boolean | character | integer |
 real | string | time. There is no standard package for textio operations on
 std_logic. Tools vendors may provide their own.

E) IEEE.NUMERIC_STD Package

type **unsigned** is array (natural range <>) of std_logic;
 type **signed** is array (natural range <>) of std_logic;
 function **shift_left** (arg : unsigned; count : natural)
 return unsigned;
 -- Other functions: **shift_right()**, **rotate_left()**, **rotate_right()**
 function **resize** (arg : signed; new_size : natural) return signed;

21. TESTBENCH

library ieee;
 use ieee.std_logic_1164.all;
 use ieee.numeric_std.all;
 entity **tb_counter** is
 end **tb_counter**;
 architecture behavioral of **tb_counter** is

```
-- Component declaration
component counter is
  port (
    clk   : in std_logic;
    reset : in std_logic;
    enable : in std_logic;
    count  : out std_logic_vector(7 downto 0) );
end component;

-- Testbench signals
signal clk   : std_logic := '0';
signal reset : std_logic := '1';
signal enable : std_logic := '0';
signal count : std_logic_vector(7 downto 0);
-- Clock period
constant clk_period : time := 10 ns;
begin
  -- Unit Under Test instantiation
  uut: counter
  port map (
    clk   => clk,
    reset => reset,
    enable => enable,
    count => count
  );
  -- Clock generation
  clk_proc: process
  begin
    clk <= '0';
    wait for clk_period/2;
    clk <= '1';
    wait for clk_period/2;
  end process;
  -- Stimulus process
  stim_proc: process
  begin
    -- Reset phase; -- End simulation
    wait;
  end process;
end behavioral;
```

21. FILES I/O FOR TESTBENCH

```
-- File I/O for testbenches
use std.textio.all;
process
  file input_file : text open read_mode is "input.txt";
  file output_file : text open write_mode is "output.txt";
  variable line_in : line;
  variable line_out : line;
  variable data : integer;
begin
  while not endfile(input_file) loop
    readline(input_file, line_in);
    read(line_in, data);
    -- Process data
    data := data * 2;
    write(line_out, data);
    writeline(output_file, line_out);
  end loop;
  wait;
end process;
```

22. FINITE STATE MACHINE

entity **moore** is
 port (clk : in bit;
 reset : in bit;
 input_x : in bit;
 output_z : out bit);

end **moore**;
 architecture behavioral of **moore** is
 type state_type is (q0,q1,q2);
 signal current_s,next_s: state_type;
 begin
 --combinational process to model next-state-logic
 process (current_s,input_x)
 begin

```
case current_s is
  when q0 => --when current state is "q0"
    if(input_x = '0') then next_s <= q0;
    else next_s <= q1;
  end if;
  when q1 => --when current state is "q1"
    if(input_x = '0') then next_s <= q1;
    else next_s <= q2;
  end if;
  when q2 => --when current state is "q2"
    if(input_x = '0') then next_s <= q1;
    else next_s <= q2;
  end if;
end case;
```

end process;
 --sequential process to describe current-state-logic
 process (clk,reset)
 begin
 if (reset='1') then
 current_s <= q0; --asynchronous reset
 elsif (clk'event and clk='1') then
 current_s <= next_s; --state change.
 end if;
end process;

process (current_s) --combinational process to model output-logic
 begin

```
case current_s is
  when q0 => output_z <= '0'; --when current state is "q0"
  when q1 => output_z <= '1'; --when current state is "q1"
  when q2 => output_z <= '0'; --when current state is "q2"
end case;
```

end process;
 end behavioral;

23. Best Practices

- Use meaningful names: enable instead of e, data_bus instead of db
- Always use OTHERS in case statements to avoid latches
- Include all inputs in sensitivity lists for combinational logic
- Use STD_LOGIC instead of BIT for flexibility
- Cast STD_LOGIC_VECTOR to UNSIGNED/SIGNED before arithmetic
- Group related signals into records for complex interfaces
- Use generics to make components reusable
- Comment your code to explain the hardware behavior

